

树链剖分 (HLD)

基本概念

树链剖分 (Heavy - Light Decomposition, HLD) 是一种将树分解为若干条链的数据结构和算法，其目的是便于在树上进行路径查询和更新操作。通过这种分解方式，能将树上的复杂路径问题转化为对链的操作，从而可以利用线段树、平衡树等数据结构高效地处理。

核心思想

树链剖分的核心思想是对树的节点进行重轻分类。对于每个节点，将其孩子中拥有最大子树的那个孩子称为重孩子，其余孩子称为轻孩子。由重孩子组成的链称为重链，轻孩子则作为新的链的起点。通过这样的划分，树上的任意路径都可分解为少数几条链，进而能利用数据结构对这些链进行高效操作。

实现步骤

相关定义

- 重儿子**：对于一个节点，其所有子节点中，子树大小最大的子节点（若有多个子树大小相同且最大的情况，任选其一）。
- 轻儿子**：除重儿子外的其他子节点。
- 重边**：连接节点与其重儿子的边。
- 轻边**：连接节点与其轻儿子的边。
- 重链**：由重边连接而成的链，即从一个节点（非重儿子）开始，通过重边依次连接重儿子所形成的链。

预处理

- 第一次 DFS**
 - 计算每个节点的深度 (dep)。
 - 记录每个节点的父节点 (fa)。
 - 计算每个节点的子树大小 (sz)。

- 确定每个节点的重儿子 (son) 。

2. 第二次 DFS

- 为每个节点分配一个新的编号 (idx) ，使同一重链上的节点编号连续，便于后续使用线段树等数据结构进行操作。
- 记录每个节点所在重链的链头 (top) 。

具体操作

1. **路径查询 / 更新**：对于树上任意两个节点 u 和 v ，将路径 $u - v$ 分解为 u 到 LCA（最近公共祖先）和 v 到 LCA 两段。在每一段中，从节点向上移动，若当前节点与链头在同一条重链上，则直接对该段进行操作；否则，操作从当前节点到链头的重链，然后移动到链头的父节点，重复上述过程，直到到达 LCA。
2. **子树查询 / 更新**：由于子树在新的编号下是连续的，因此可以直接通过子树的根节点的编号和子树大小确定查询 / 更新的区间。

视频讲解

LCA

{{< bilibili BV1tY4y1G7em >}}

轻重链

{{< bilibili BV18S4y1e7qb >}}

应用场景

1. 树上路径的最值查询、求和、更新等操作。
2. 解决与树的直径、LCA 相关的复杂问题。
3. 在一些图论问题中，将树结构转化为链结构后进行处理。

两次DFS的代码实现

```

int pre_wealth[N],dep[N],sz[N],son[N],fa[N],top[N],idx[N],nw[N],cnt;
vector<int> g[N];
void depdfs(int u,int father)
{
    fa[u] = father,dep[u]=dep[father]+1,sz[u] = 1;
    for(auto v : g[u])
    {
        if(v==father) continue;
        depdfs(v,u);
        sz[u] += sz[v];
        if(sz[son[u]]<sz[v]) son[u] = v;
    }
}

void dfs(int u,int t)
{
    top[u] = t;
    idx[u] = ++cnt;
    nw[cnt] = pre_wealth[u];
    if(!son[u]) return;
    dfs(son[u],t);
    for(auto v : g[u])
    {
        if(v==fa[u] or v==son[u]) continue;
        dfs(v,v);
    }
}

```

寻找LCA的应用

基本题

P3379 【模板】最近公共祖先 (LCA)

P5903 【模板】树上 K 级祖先

拓展题——[苹果树]:

问题描述:

给出一棵包含 n 个点的树，每个节点 i 都有一个权值 a_i 。

有 m 次操作，每次操作都形如以下的两种：

- 1 x y：查询 x 到 y 的路径上，最大的点权权值。

2. $2 \times z$: 对于所有与 x 直接相连的点 i , 令 $a_i \leftarrow a_i + z$ 。

不保证查询的 x, y 满足 $x \neq y$ 。

输入:

每个测试点中包含多组测试数据。输入的第一行包含一个正整数 T ($1 \leq T \leq 110$) , 表示数据组数。对于每组测试数据:

- 第一行两个正整数 n, m ($1 \leq n, m \leq 10^5$) 。
- 第二行 n 个整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^4$) , 表示初始每个节点的点权。
- 接下来 $n - 1$ 行, 每行两个正整数 x, y ($1 \leq x, y \leq n$) , 表示有一条从 x 到 y 的无向边。
- 接下来 m 行, 每行三个整数, 第一个数 opt 表示操作类型:
 - 若 $opt = 1$, 则后面两个数 x, y ($1 \leq x, y \leq n$) , 表示询问路径。
 - 若 $opt = 2$, 则后面两个数 x, z ($1 \leq x \leq n, 1 \leq z \leq 10^4$) , 分别表示修改中心以及增加的值。保证所有测试数据中 n 之和与 m 之和均不超过 4×10^5 。

输出:

对于每组测试数据: 对于每一个 $opt = 1$ 的操作, 输出一行一个数表示答案。

样例输入:

```
1
5 10
3 7 9 1 6
2 1
3 1
4 2
5 4
2 1 2
2 5 3
1 1 4
1 3 1
2 4 3
2 2 9
2 1 5
1 4 2
2 3 4
1 4 4
```

样例输出:

```
9
11
17
13
```

思路:

采用重链剖分结合线段树解决树上路径最大权值查询与邻接点更新问题。

- 先通过两次 DFS 完成重链剖分，将树分解为若干重链，使每条链节点在线段树中对应连续区间。
- 用线段树维护重链节点权值，支持区间查询和单点更新。
- 对节点邻接点更新，重儿子和父节点通过线段树更新，轻儿子用 flag 数组记录累计更新。
- 路径查询时，分解路径为多个重链区间，查各区间最大值并结合 flag 数组补充轻儿子实际权值，取最大结果。
-

STD

```

// #pragma GCC optimize(2)
#include <bits/stdc++.h>
#define endl "\n"
#define int long long
using namespace std;
const int N = 1e5+10;
const int inf = 1e18+10;

struct node {
    int data, lazy = 0;
};

void build(const vector<int> &data, vector<node> &tree, int rt, int left, int right) {
    tree[rt].lazy = 0;
    if (left == right) {
        tree[rt].data = data[left]; // 叶子节点
        return;
    }
    int mid = left + (right - left) / 2;
    build(data, tree, 2*rt, left, mid); // 递归构建左子树
    build(data, tree, 2*rt + 1, mid + 1, right); // 递归构建右子树
    tree[rt].data = max(tree[2 * rt].data, tree[2 * rt + 1].data); // 更新当前节点的值
}

void pushDown(vector<node> &tree, int rt) {
    if (tree[rt].lazy != 0) {
        // 更新左子树
        tree[2 * rt].data += tree[rt].lazy;
        tree[2 * rt].lazy += tree[rt].lazy;
        // 更新右子树
        tree[2 * rt + 1].data += tree[rt].lazy;
        tree[2 * rt + 1].lazy += tree[rt].lazy;
        // 清空当前节点的懒标记
        tree[rt].lazy = 0;
    }
}

int query(vector<node> &tree, int rt, int left, int right, int ql, int qr) {
    if (ql > right || qr < left) {
        return -inf; // 若区间无交集，返回最小值，表明该区间对结果无贡献
    }
    if (ql <= left && qr >= right) {
        return tree[rt].data; // 若完全包含，直接返回当前节点的值
    }
    int mid = left + (right - left) / 2;
    pushDown(tree, rt); // 若当前节点有延迟标记，将其下推到子节点
    int leftMax = query(tree, 2*rt, left, mid, ql, qr); // 递归查询左子树
    int rightMax = query(tree, 2*rt+1, mid+1, right, ql, qr); // 递归查询右子树
    return max(leftMax, rightMax);
}

```

```

void update(vector<node> &tree, int rt, int left, int right, int idx, int value) {
    if(left == right){
        tree[rt].data += value;
        tree[rt].lazy = 0; //找到目标节点, 更新值
        return;
    }
    int mid = left + (right - left) / 2;
    pushDown(tree, rt); //处理懒标记
    if(idx <= mid) update(tree, 2*rt, left, mid, idx, value);
    else update(tree, 2*rt+1, mid+1, right, idx, value);
    tree[rt].data = max(tree[rt*2].data, tree[rt*2+1].data);
}

int n,m;
int pre_wealth[N];
vector<int> g[N];
vector<node> tree;
vector<int> nw(N,0);
int dep[N],sz[N],son[N],fa[N],top[N],idx[N],cnt;

int flag[N];
void depdfs(int u,int father)
{
    fa[u] = father,dep[u]=dep[father]+1,sz[u] = 1;
    for(auto v : g[u])
    {
        if(v==father) continue;
        depdfs(v,u);
        sz[u] += sz[v];
        if(sz[son[u]]<sz[v]) son[u] = v;
    }
}

void dfs(int u,int t)
{
    top[u] = t;
    idx[u] = ++cnt;
    nw[cnt] = pre_wealth[u];
    if(!son[u]) return;
    dfs(son[u],t);
    for(auto v : g[u])
    {
        if(v==fa[u] or v==son[u]) continue;
        dfs(v,v);
    }
}

int query_path(int u,int v)
{
    int res = -inf;

```

```

while(top[u]!=top[v])
{
    if(dep[top[u]]<dep[top[v]]) swap(u,v);
    if(fa[u]!=0 and son[fa[u]]!=u)
        res = max(res,query(tree,1,1,n,idx[u],idx[u])+flag[fa[u]]);
    if(fa[v]!=0 and son[fa[v]]!=v)
        res = max(res,query(tree,1,1,n,idx[v],idx[v])+flag[fa[v]]);
    res = max(res,query(tree,1,1,n,idx[top[u]],idx[u]));
    if(fa[top[u]]!=0 and son[fa[top[u]]]!=top[u])
        res = max(res,query(tree,1,1,n,idx[top[u]],idx[top[u]]+flag[fa[top[u]]]));
    u = fa[top[u]];
}
if(dep[u]<dep[v]) swap(u,v);
if(fa[u]!=0 and son[fa[u]]!=u)
    res = max(res,query(tree,1,1,n,idx[u],idx[u])+flag[fa[u]]);
if(fa[v]!=0 and son[fa[v]]!=v)
    res = max(res,query(tree,1,1,n,idx[v],idx[v])+flag[fa[v]]);
res = max(res,query(tree,1,1,n,idx[v],idx[u]));
return res;
}

void update_path(int u,int z)
{
    flag[u]+=z;
    if(son[u]!=0) update(tree,1,1,n,idx[son[u]],z);
    if(fa[u]!=0) update(tree,1,1,n,idx[fa[u]],z);
}

void solve()
{
    cin >> n >> m;
    cnt = 0;
    tree.clear();
    tree.resize(4 * n + 5);
    for (int i = 1; i <= n; i++)
    {
        cin >> pre_wealth[i];
        g[i].clear();
        dep[i] = sz[i] = son[i] = fa[i] = top[i] = idx[i] = nw[i] = flag[i] = 0;
    }
    for(int i=1;i<=n-1;i++)
    {
        int u,v;cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    depdfs(1,0);
    dfs(1,1);
    build(nw,tree,1,1,n);
    for(int i=1;i<=m;i++)
    {

```



```

int op;cin >> op;
if(op==1)
{
    int x,y;cin >> x >> y;
    cout << query_path(x,y) << endl;
}
else
{
    int x,z;cin >> x >> z;
    update_path(x,z);
}
}

}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0),cout.tie(0);
    int _ = 1;
    cin >> _;
    while(_--)
    {
        solve();
    }
    return 0;
}

```

HLD模板代码

```

struct HLD
{
    int n;
    vector<vector<int>> g;
    vector<int> dep, sz, son, fa, top, idx, out, HLD_data;
    int cnt;
    HLD() {}
    HLD(int _n) { init(_n); }
    void init(int _n) {
        n = _n;
        g.resize(n+1, {});
        dep.resize(n+1, 0);
        sz.resize(n+1, 0);
        son.resize(n+1, 0);
        fa.resize(n+1, 0);
        top.resize(n+1, 0);
        idx.resize(n+1, 0);
        out.resize(n+1, 0);
        HLD_data.resize(n+1, 0);
        cnt = 0;
    }
    void addEdge(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }
    void prework(int root = 1)
    {
        cnt = 0;
        depdfs(root, 0);
        dfs(root, root);
    }
    // 第一次 DFS: 计算 sz, dep, son, fa
    void depdfs(int u, int father)
    {
        fa[u] = father;
        dep[u] = dep[father] + 1;
        sz[u] = 1;
        son[u] = 0;
        for (int v : g[u])
        {
            if (v == father) continue;
            depdfs(v, u);
            sz[u] += sz[v];
            if (sz[v] > sz[son[u]]) son[u] = v;
        }
    }
    // 第二次 DFS: 分链, 给 idx, top, out 赋值
    void dfs(int u, int t)
    {

```

```

top[u] = t;
idx[u] = ++cnt;
HLD_data[cnt] = u;
// 先重儿子
if (son[u]) dfs(son[u], t);
// 再轻儿子
for (int v : g[u])
{
    if (v == fa[u] || v == son[u]) continue;
    dfs(v, v);
}
// 无论是不是叶子，都在此处给 out[u] 赋值
out[u] = cnt;
}
int query_lca(int u, int v)
{
    while (top[u] != top[v])
    {
        if (dep[top[u]] < dep[top[v]]) swap(u, v);
        u = fa[top[u]];
    }
    return dep[u] < dep[v] ? u : v;
}
int query_klca(int u, int k)
// 求 x 的第 k 个祖先 (0 表示 x 本身)
{
    if (dep[u] < k) return -1;
    int fa_dep = dep[u] - k;
    // 跳到那条重链的顶，直到该链顶部深度 ≤ fa_dep
    while (dep[top[u]] > fa_dep) u = fa[top[u]];
    // 链内直接跳：当前节点在序 idx[u]，它的深度是 dep[u]，
    // 要到深度 fa_dep 的节点，走 idx 减去 dep[u]-fa_dep
    return HLD_data[idx[u] - (dep[u] - fa_dep)];
}
bool isAncestor(int u, int v)
{
    // [idx[u], out[u]] 包含所有子节点
    return idx[u] <= idx[v] && idx[v] <= out[u];
}
int dist(int u, int v)
{
    return dep[u] + dep[v] - 2 * dep[query_lca(u, v)];
}
int rootedParent(int u, int v)
// 在以 u 为根的树中，查询 v 的父节点（朝向 u 的方向）
{
    swap(u, v);
    if (u == v) return u;
    // 若 u 不是 v 的祖先，则 u 的父亲就是答案
    if (!isAncestor(u, v)) return fa[u];
    // 否则在 u 的孩子里，找一条链带 v 下去的那位

```

```

        auto it = upper_bound(g[u].begin(), g[u].end(), v, [&](int a, int b) {
            return idx[a] < idx[b];
        }) - 1;
        return *it;
    }
    int rootedSize(int u, int v)
    // 在以 u 为根的树中, 计算 v 子树的大小
    {
        if (u == v) return n;
        // 如果 v 不是 u 的子孙, 则就是普通子树大小
        if (!isAncestor(v, u)) return sz[v];
        // 如果 v 是 u 的祖先, 则去掉通向 u 那条孩子的子树
        return n - sz[rootedParent(u, v)];
    }
    int rootedLca(int a, int b, int c)
    // 三点 LCA: 利用性质 LCA(a,b,c)=LCA(a,b)^LCA(a,c)^LCA(b,c)
    {
        return query_lca(a, b) ^ query_lca(a, c) ^ query_lca(b, c);
    }
    /*配合线段树使用
    int query_path(int u,int v)
    {
        int res = -inf;
        while(top[u]!=top[v])
        {
            if(dep[top[u]]<dep[top[v]]) swap(u,v);
            // if(fa[u]!=0 and son[fa[u]]!=u)
            // res = max(res,query(tree,1,1,n,idx[u],idx[u])+flag[fa[u]]);
            // if(fa[v]!=0 and son[fa[v]]!=v)
            // res = max(res,query(tree,1,1,n,idx[v],idx[v])+flag[fa[v]]);
            res += query(tree,1,1,n,idx[top[u]],idx[u]);
            // if(fa[top[u]]!=0 and son[fa[top[u]]]!=top[u])
            // res =
            max(res,query(tree,1,1,n,idx[top[u]],idx[top[u]])+flag[fa[top[u]]]);
            u = fa[top[u]];
        }
        if(dep[u]<dep[v]) swap(u,v);
        // if(fa[u]!=0 and son[fa[u]]!=u)
        // res = max(res,query(tree,1,1,n,idx[u],idx[u])+flag[fa[u]]);
        // if(fa[v]!=0 and son[fa[v]]!=v)
        // res = max(res,query(tree,1,1,n,idx[v],idx[v])+flag[fa[v]]);
        res += query(tree,1,1,n,idx[v],idx[u]);
        return res;
    }
    */
};

```